MetaMove — Technical Documentation

Distance-Based Speed Scaling · Pinch-and-Move Teleoperation · Dashboard & HMI

Elias Bitsch

Jun 25, 2026

Table of Contents

MetaMove is a mixed-reality teleoperation system for an **ABB GoFa CRB 15000** collaborative robot. An operator wearing a **Meta Quest 3** headset sees the real robot through passthrough, controls its end-effector with bare-hand gestures, and is protected by a distance-based speed-scaling safety layer. A web dashboard and an in-headset HMI provide monitoring and supervisory control.

This document describes the three core subsystems:

1. **Distance-Based Speed Scaling** — the robot slows down and freezes as a human approaches, and ramps back up smoothly when they retreat.
2. **Pinch-and-Move End-Effector Control** — the operator pinches a virtual handle on the robot flange and drags it through space; the motion is solved to joint commands and streamed to the controller.
3. **Dashboard and HMI** — a FastAPI/WebSocket web dashboard plus a layered Unity in-headset interface, and a set of operator console tools.

Chapter 1

Installation & Setup

This chapter covers hardware and software requirements, dependencies, and the bring-up sequence for the full Quest → ROS 2 → GoFa pipeline. You can run individual subsystems standalone (e.g. the dashboard against a simulated robot) — each section notes its own prerequisites.

1.1 Hardware requirements

Component	Requirement
Robot	ABB GoFa CRB 15000 (5 kg / 950 mm), RobotWare 7.x with the **EGM** and **Externally Guided Motion** options.
Robot controller	OmniCore, reachable over Ethernet; RWS (HTTPS) enabled.
Headset	Meta Quest 3 (passthrough + hand tracking), developer mode enabled.
Workstation	Windows 11 PC on the robot LAN; dedicated wired NIC for the robot subnet.
Power	Use a sufficiently rated PSU for the workstation. A 130 W supply browns out under combined load — prefer a USB-A→USB-C cable for `adb` to avoid USB-PD draw on the laptop.
Network	Isolated robot LAN (e.g. `10.x` / a dedicated `/24`). The controller, the Windows EGM bridge, and the workstation must share that subnet.

1.2 Software requirements

Layer	Software
ROS 2	**Jazzy** (run in Docker on **WSL2**, not Docker Desktop).
Motion	MoveIt 2 (+ MoveIt Servo), `ros2_control`, `joint_trajectory_controller`.
Bridge	`rosbridge_suite` (WebSocket on `:9090`), ROS-TCP-Endpoint (`:10000`).
Unity	Unity 6 LTS with Meta XR SDK (v85), ROS-TCP-Connector.
EGM bridge	Python 3.11+ on Windows (`roslibpy`, `numpy`).
Dashboard	Python 3.11+ (`fastapi`, `uvicorn`, `rclpy`); optional PostgreSQL.
Docs	Python 3.10+ with `sphinx`, `myst-parser`, `furo`, `rinohtype` (this document).

1.3 Network configuration

The pipeline depends on a correctly configured robot subnet. Key points:

- Give the Windows EGM bridge a **fixed IP on the robot subnet** and bind the bridge socket to that address — **never bind to `0.0.0.0`** on a multi-homed host. The controller's UDP unicast device

(UDPUC) silently discards correction packets that arrive from an unexpected source IP.

- Configure the controller UDPUC device RemoteAddress to the bridge IP and the matching port (:6511 for joint, :6515 for the pose/ROB_1 path).
- On WSL2, forward the rosbridge (9090) and ROS-TCP (10000) ports with netsh portproxy so the Quest can reach the ROS graph through the Windows host.
- Identify the **physical** robot NIC (not a loopback/virtual adapter) before assigning the static IP.

Note RWS access as the default controller user is limited: writing PERS variables works, but program execution, configuration changes, and file upload do not. Use the FlexPendant for config changes, or load RAPID modules via the documented RWS 2.0 loadmod / PP-reset recipe.

1.4 Bring-up sequence (real robot)

1. **Network** — power the controller, verify the workstation can ping it and the assigned bridge IP exists on the robot NIC.
2. **RAPID** — ensure the EGM RAPID module (MetaJointMain / equivalent) is the active task and the program pointer is set to its main routine. Motors on.
3. **ROS 2** — start the Docker stack in WSL2 (MoveIt, controllers, rosbridge, ROS-TCP-Endpoint, and the metamove_bridge nodes).
4. **EGM bridge (Windows)** — start egm_bridge_servo.py, bound to the bridge IP, pointing at the controller UDPUC port. Confirm /joint_states goes live.
5. **MoveIt Servo** — activate servo (activate_servo.py) and confirm /servo_node/status is nominal.
6. **Quest app** — launch the MetaMove app; it connects via ROS-TCP-Connector. Calibrate the QR /world anchor so the virtual robot overlays the real one.
7. **Safety check** — verify distance scaling: walking toward the robot must reduce speed and freeze it; retreating must ramp it back up.

Warning Always validate new RAPID/EGM features on a RobotStudio Virtual Controller first, then on the real robot. Keep acceleration limits conservative (e.g. accel ~ 0.10) during bring-up.

1.5 Subsystem-local setup

1.5.1 ROS 2 bridge package

The metamove_bridge package (ros2/docker/metamove_bridge/) is a standard ament_python package. Inside the ROS 2 container:

```
cd /ros2_ws
colcon build --packages-select metamove_bridge
source install/setup.bash
ros2 launch metamove_bridge bringup.launch.py   # or run individual nodes
```

1.5.2 EGM bridge & operator consoles (Windows)

```
cd bridge\egm-bridge
python -m venv .venv ; .\.venv\Scripts\Activate.ps1
```

```
pip install roslibpy numpy
python egm_bridge_servo.py --rosbridge-host <ROBOT_LAN_IP>
```

The console tools (`control_console.py`, `teleop_console.py`, `playback_console.py`, `relay_speed_console.py`, `rescue_jog.py`, …) connect to `rosbridge` on `:9090`. See Dashboard and HMI for what each one does.

1.5.3 Web dashboard

```
cd MRE2-GOFA_Dashboard/backend
pip install -r requirements.txt
uvicorn app:app --host 0.0.0.0 --port 8080
```

Or use the slim container under `deploy/dashboard/` to join an existing ROS 2 graph. The dashboard auto-discovers ROS topics at runtime.

1.5.4 Unity Quest client

1. Open `unity-quest/` in Unity 6 LTS with the Meta XR SDK and ROS-TCP-Connector installed (see `Packages/manifest.json`).
2. Set the ROS-TCP-Connector endpoint to the Windows host IP / port `10000`.
3. Build to Android (Quest), deploy with `adb install`. Use a USB-A→USB-C cable to avoid the USB-PD brownout issue noted above.

1.6 Building this documentation

The documentation source lives in `docs/technical/`. To build it yourself:

```
cd docs/technical
pip install -r requirements.txt

# HTML (browsable)
sphinx-build -b html . _build/html

# PDF (pure-Python via rinohtype - no LaTeX toolchain required)
sphinx-build -b rinoh . _build/pdf
# -> _build/pdf/MetaMove-Technical-Documentation.pdf
```

`requirements.txt` pins `sphinx`, `myst-parser`, `furo`, and `rinohtype`.

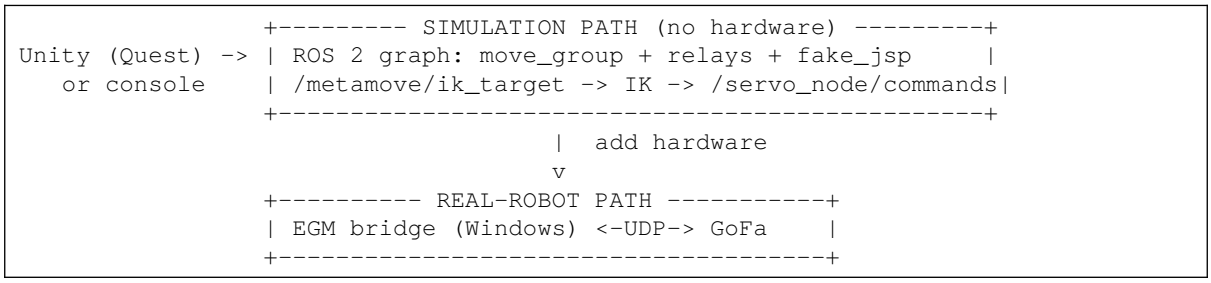
Chapter 2

Reproduction Guide — Build It Yourself

This chapter is a complete, ordered recipe for rebuilding MetaMove from scratch. It is written so that the **simulation path needs no robot and no headset** — you can stand up the full ROS 2 motion stack on one Linux/WSL2 machine and exercise the IK loop, the playback, and the distance speed scaler against a fake joint-state publisher. The **real-robot path** then adds the RAPID module, the controller `UDPUC` config, the Windows EGM bridge, and the Quest client.

All exact values below (node names, parameters, topics, ports) are taken from the source in this repository. Substitute your own LAN addresses where you see `<...>`; the lab's own values are shown as concrete examples.

2.1 0. What you are building



Two interchangeable IK backends exist; pick one:

- `moveit_ik_relay` — calls MoveIt `/compute_ik`, outputs joint positions.
- `pose_to_twist_node` + MoveIt Servo — outputs a Cartesian twist.

2.2 1. Prerequisites

Tool	Version / note
OS	Linux or Windows 11 + **WSL2** (Ubuntu). Run ROS in Docker **in WSL2**, not Docker Desktop.
Docker	Engine with Compose v2.

ROS 2	**Jazzy** (provided by the image; you do not install it on the host).
Robot description	`abb_crb15000_moveit` MoveIt config package for the GoFa CRB 15000 5/0.95.
Unity	**6000.4.0f1** (Unity 6 LTS) — only for the headset client.
Robot	ABB GoFa CRB 15000, RobotWare 7.x with the EGM option — only for the real path.

2.3 2. Build the ROS 2 image

The image is defined by `ros2/docker/Dockerfile` (base `ros:jazzy-ros-base`). It installs MoveIt + MoveIt Servo, `ros2_control`, `rosbridge_suite`, `ros-tcp-endpoint`, and `octomap`, and builds the `metamove_bridge` package into `/opt/metamove_ws`.

```
cd MetaMove
docker build -f ros2/docker/Dockerfile -t metamove/ros2:jazzy .
```

Key environment baked into the image (override at `docker run` as needed):

```
ROS_DOMAIN_ID=42
RMW_IMPLEMENTATION=rmw_fastrtps_cpp
METAMOVE_RWS_IP=192.168.125.1   METAMOVE_RWS_PORT=443   METAMOVE_EGM_PORT=6511
```

The `metamove_bridge` package (`ament_python`) exposes these console scripts (`ros2 run metamove_bridge <name>`):

```
bridge_node             moveit_ik_relay         dpp_teach
pose_to_twist_node      jtc_servo_relay         dpp_playback
fake_joint_state_publisher   distance_speed_scaler   dpp_orchestrate
fake_cloud_publisher    jtc_egm_stub            dpp_gui
```

2.4 3. Simulation path (no robot, no headset)

This proves the whole motion core on one machine. The fake joint-state publisher closes the loop by integrating `/servo_node/commands` back into `/joint_states`.

2.4.1 3a. IK loop

```
docker run -it --net host -v "$PWD/ros2:/opt/metamove_ws/src" metamove/ros2:jazzy
bash
# inside the container:
source /opt/ros/jazzy/setup.bash && source /opt/metamove_ws/install/setup.bash
ros2 launch metamove_bridge metamove_sim_ik.launch.py
```

This launch starts, in order: `robot_state_publisher` (50 Hz), `move_group` (provides `/compute_ik`), `rosbridge_websocket` (`:9090`), `ros_tcp_endpoint` (`:10000`), `fake_joint_state_publisher`, and `moveit_ik_relay`.

Drive it without Unity by publishing a target pose:

```
ros2 topic pub -r 50 /metamove/ik_target geometry_msgs/PoseStamped \
  '{header: {frame_id: "base_link"}, pose: {position: {x: 0.4, y: 0.0, z: 0.5},
    orientation: {x: 0, y: 1, z: 0, w: 0}}}'
ros2 topic echo /servo_node/commands      # watch joint commands appear
```

`moveit_ik_relay` (node `moveit_ik_relay`, 50 Hz) gates on target freshness (`target_timeout=0.3`) and seed freshness (`joint_state_timeout=0.5`), seeds `/compute_ik` (`group_name="manipulator"`, 50 ms IK timeout, `avoid_collisions=True`) from

`/joint_states`, slew-limits each joint by `max_joint_speed=0.3` rad/s, and publishes `/servo_node/commands` (`std_msgs/Float64MultiArray`, joints in rad).

2.4.2 3b. Waypoint playback + distance speed scaling

```
ros2 launch metamove_bridge metamove_sim_playback.launch.py
```

This starts `jtc_servo_relay` (rclpy node name **`joint_trajectory_controller`**, `time_scale=2.0`, `rate_hz=50`, `live_speed=1.0`) and `dpp_playback` (`velocity_scaling=0.5`, `acceleration_scaling=0.5`, `dwell_seconds=0.5`, `waypoints_file=.../dpp_waypoints.yaml`). Teach waypoints into the YAML with `ros2 run metamove_bridge dpp_teach`; each entry is `{joints: [j1..j6]}` in radians.

Add the safety layer:

```
ros2 run metamove_bridge distance_speed_scaler
# simulate an approaching human:
ros2 topic pub -r 20 /quest/min_distance std_msgs/Float32 '{data: 0.5}'   # <
d_near -> freeze
ros2 topic pub -r 20 /quest/min_distance std_msgs/Float32 '{data: 1.3}'   #
mid-band -> partial
ros2 topic echo /robot/speed_factor
```

`distance_speed_scaler` (node `distance_speed_scaler`, 20 Hz) maps distance through the band `d_near=0.6 .. d_far=2.0`, smooths it (`ema_alpha=0.3`), ramps up at `up_rate=0.6` /s, drops instantly, and writes the `live_speed` parameter of `joint_trajectory_controller` via `/joint_trajectory_controller/set_parameters`. It also pauses/resumes `dpp_playback` on stale distance (`stale_timeout=1.5`).

2.4.3 3c. Servo-twist backend (alternative to 3a)

If you prefer velocity control through MoveIt Servo, run `pose_to_twist_node` (node `pose_to_twist`) instead of the IK relay. It reads `/metamove/ik_target`, looks up `base_link -> tool0` via TF, applies `linear_gain=1.5` / `angular_gain=1.5` with clamps `max_linear=0.25` m/s, `max_angular=1.0` rad/s, deadbands `0.002` m / `0.01` rad (`track_orientation=False` by default), and publishes `/servo_node/delta_twist_cmds` (`geometry_msgs/TwistStamped`, 100 Hz).

2.5 4. Unity Quest client

2.5.1 4a. Project setup

Create a Unity **6000.4.0f1** project and add (see `unity-quest/Packages/manifest.json`):

Package	Version
`com.meta.xr.sdk.all`	85.0.0
`com.unity.robotics.ros-tcp-connector`	git `main`
`com.unity.robotics.urdf-importer`	git `main`
`com.unity.xr.interaction.toolkit`	3.4.1
`com.unity.xr.hands`	1.7.3
`com.unity.render-pipelines.universal`	17.4.0

Open `Assets/MetaMove/Scenes/Scene_Robot.unity` (the teleop scene). `Hud.unity` is the standalone HUD; `Scene_QRTest.unity` is QR-anchor calibration.

2.5.2 4b. ROS connection

Add a `RosBridgeBootstrap` GameObject (`Assets/MetaMove/Scripts/Robot/Ros/RosBridgeBootstrap.cs`). Set `rosIPAddress` to the host running `ros_tcp_endpoint` (lab default `192.168.125.99`, or `localhost` when the editor runs on the same PC) and `rosPort = 10000`.

2.5.3 4c. Wire the feature scripts (Inspector)

`IKTargetPosePublisher` — `topic="/metamove/ik_target"`, `frameId="base_link"`, `publishHz=50`, `onlyWhenGrabbed=true`; assign `target` (the IK target ball) and `robotBase` (the QR-anchored `base_link`). Publishes `geometry_msgs/PoseStamped` after converting Unity left-handed Y-up to ROS FLU.

`PhantomGrabRelay` (on the end-effector sphere) — assign `grabbable`, `ikTarget`, and a hand `handAnchor`; `dragGain=1.0`, `maxReachM=0.5`.

`SafetyHud` — assign `robotBase` and `humanPoints` (defaults to `Camera.main`); `useRos=true`, `distanceTopic="/quest/min_distance"`, `speedTopic="/robot/speed_factor"`, `publishHz=20`, thresholds `dangerDist=0.6`, `warnDist=1.2`, `speedDistNear=0.6`, `speedDistFar=2.0`.

`ScalingModeToggle` — `topic="/quest/scaling_enabled"`, `scalingEnabled=true`, `vrButton=OVRInput.Button.Two`; publishes `std_msgs/Bool` on a 2 s heartbeat.

2.5.4 4d. Build & deploy

Build to Android and install on the Quest:

```
adb install -r MetaMove.apk
```

Use a **USB-A -> USB-C** cable for `adb`, not USB-C -> USB-C: USB-PD negotiation can brown out / crash the laptop on the 130 W supply.

2.6 5. Real-robot path

> **Warning** Do every new RAPID/EGM step on a RobotStudio Virtual Controller first, then the real robot. Keep acceleration conservative during bring-up. The robot moves under external command — be ready on the E-Stop.

2.6.1 5a. RAPID module

Load the joint-streaming module (`robotstudio/rapid/MetaMoveJointStreamFix.mod`) and make it the active task with the PP at its main routine. Its EGM cycle is:

```
EGMSetupUC ROB_1, egmId, "default", "ROB_1" \Joint;
EGMActJoint egmId
    \Tool:=tool0
    \J1:=egmMM \J2:=egmMM \J3:=egmMM
    \J4:=egmMM \J5:=egmMM \J6:=egmMM
    \LpFilter:=20 \SampleRate:=4
    \MaxSpeedDeviation:=1000;
EGMRunJoint egmId, EGM_STOP_HOLD
    \J1 \J2 \J3 \J4 \J5 \J6
    \CondTime:=600 \RampInTime:=0.05;
```

Note Do **not** pass `\PosCorrGain` to `EGMActJoint` on RW 7.20 — it raises elog 40160. Joint correc-

tions need no gain. `MaxSpeedDeviation` here is the dominant speed cap and overrides the ROS-side `live_speed`.

2.6.2 5b. Controller UDPUC device

Configure the EGM unicast device so its `RemoteAddress` is the **bridge's** IP and the port matches (`6511` joint / `6515` pose):

```
UDPUC:
  -Name "ROB_1" -Type "UDPUC" \
  -RemoteAddress "192.168.125.150" \
  -RemotePortNumber 6511 -LocalPortNumber 6511
```

Set this on the FlexPendant (the default RWS user cannot write CFG). The single most common failure is a **source-IP mismatch**: the controller silently drops any correction packet whose source IP is not the configured `RemoteAddress`.

2.6.3 5c. Windows EGM bridge

```
cd bridge\egm-bridge
python -m venv .venv ; .\.venv\Scripts\Activate.ps1
pip install roslibpy numpy
# BIND to the robot-subnet IP - never 0.0.0.0 on a multi-homed host:
python egm_bridge_servo.py --host 192.168.125.150 --port 6511 --rosbridge-host
192.168.125.99
```

The bridge (`egm_bridge_servo.py`) opens a UDP socket bound to `--host:--port`, parses the controller's `EgmRobot` feedback (joints in degrees, ~250 Hz), publishes `/joint_states` (deg->rad) at ~50 Hz, subscribes `/servo_node/commands` (rad) over rosbridge, converts rad->deg, and sends `EgmSensor` correction packets (`MSGTYPE_CORRECTION`). If no command arrives within 0.5 s it echoes feedback (identity mode) to keep the EGM session alive.

Note If you run the bridge inside a container under WSL2 with mirrored networking, inbound UDP may never reach the container namespace. Use a **macvlan** network so the container holds a real address on the robot subnet (e.g. `192.168.125.150/24`), or run the bridge natively on Windows.

2.6.4 5d. Activate servo & verify

```
python activate_servo.py --rosbridge-host 192.168.125.99   # start_servo + TWIST
mode
```

Verification checklist:

- Bridge stdout shows ~250 packets/s received from the controller.
- `ros2 topic hz /joint_states` ~ 50 Hz.
- `ros2 topic echo /servo_node/status` is nominal (no collision/singularity stop).
- Publishing `/metamove/ik_target` (or jogging in the Quest) moves the robot.
- Walking toward the robot reduces `/robot/speed_factor` and freezes motion; retreating ramps it back up.

2.7 6. Troubleshooting

Symptom	Cause	Fix
Robot ignores EGM corrections	Bridge sends from wrong source IP	Bind the socket to the UDPUC `RemoteAddress`; never `0.0.0.0`.
`elog 40160` on EGMActJoint	`\PosCorrGain` used in joint mode on RW 7.20	Remove it — joint corrections take no gain.
Container receives 0 UDP packets	WSL2 mirrored-networking bug	Use a macvlan network or run the bridge on Windows.
IK relay never commands	No fresh `/joint_states` seed	Start the bridge / fake JSP first; check `joint_state_timeout`.
Robot jumps at start	Singular seed	Seed near `[0, 0, -0.785, 0, -0.785, 0]` (the fake JSP default).
Speed never reaches 100%	Distance below `d_far`, or `MaxSpeedDeviation` cap	Move past 2.0 m; raise the RAPID `MaxSpeedDeviation` cap.
`adb` crashes the laptop	USB-PD draw over USB-C	Use a USB-A -> USB-C cable.

Chapter 3

Distance-Based Speed Scaling

3.1 Purpose

The robot's motion speed is continuously scaled by the distance between the nearest human and the robot. As the operator approaches, the robot slows down; below a near threshold it freezes entirely. As they retreat, speed ramps back up smoothly. This is a collaborative-safety layer that runs **on top of** any commanded motion — teach playback, pinch teleoperation, or autonomous pick-and-place.

The design has two defining properties:

- **Asymmetric response** — speed ramps **up slowly** (smooth acceleration) but drops to a stop **instantly** (reactive safety).
- **Stale-safe** — if the distance signal goes stale (e.g. the headset is removed), the system freezes / pauses cleanly rather than coasting.

There are two implementations sharing the same formula: a **local** Unity-only path for offline demos, and the **ROS** path used with the real robot.

3.2 Data flow

```
Quest human pose (head + optional hands)
        |
        +-- LOCAL DEMO ------------------------------------------------+
        |   ProximitySpeedController.cs                                 |
        |     min_distance(human -> robotBase) -> EMA filter -> slew     |
        |     -> Factor (0..1) -> PickPlaceLoop motion gain             |
        |                                                               |
        +-- ROS PATH --------------------------------------------+  |
            SafetyHud.cs                                        |  |
              publishes /quest/min_distance  (Float32, ~20 Hz)  |  |
                        |                                       |  |
                        v                                       |  |
            distance_speed_scaler.py  (ROS 2, 50 Hz)            |  |
              subscribe /quest/min_distance                     |  |
              EMA filter + asymmetric slew + stale timeout      |  |
              +- publish /robot/speed_factor   (heartbeat -> HUD)   |  |
              +- set param jtc_servo_relay.live_speed  (0..1)       |  |
                        |                                       |  |
                        v                                       |  |
            jtc_servo_relay.py  (50 Hz)                         |  |
              tc += period * live_speed   (advance time cursor)     |  |
```

```
            interpolate keyframes -> /servo_node/commands          |  |
                       |                                          |  |
                       v                                          v  v
          EGM bridge (Windows) -> UDP -> GoFa controller
```

3.3 The scaling law

The distance `d` (metres) is mapped to a raw factor with a linear band between a near and far threshold:

```
band(d) = 0.0                          if d <= d_near       (freeze)
band(d) = (d - d_near)/(d_far - d_near) if d_near < d < d_far
band(d) = 1.0                          if d >= d_far        (full speed)
```

The raw factor is then **low-pass filtered** (exponential moving average) and passed through an **asymmetric slew limiter**:

```
filtered = alpha * raw + (1 - alpha) * filtered          # EMA smoothing
if raw_distance <= d_near: target = 0             # hard safety overrides filter

# asymmetric slew
if target > v_out:  v_out = min(target, v_out + up_rate * dt)   # ramp up slowly
else:               v_out = target                              # drop instantly
```

If no fresh distance arrives within the stale timeout, the output collapses to `0` and, on the ROS path, the playback node is paused for a clean stop instead of a hard jerk.

3.3.1 Default parameters

Parameter	Default	Meaning
`d_near`	0.6 m	At/below this distance the factor is 0 (freeze).
`d_far`	2.0 m	At/above this distance the factor is 1 (full speed).
`ema_alpha` (alpha)	0.3	Low-pass strength; higher = more responsive, less smooth.
`up_rate`	0.6 / s	Maximum rise of the factor per second (smooth acceleration).
`stale_timeout`	1.5 s	No distance for this long -> freeze / pause.
`min_delta`	0.01	Skip param writes smaller than this.
`max_rate_hz`	15.0	Cap on `live_speed` write rate to the relay.
scaler tick	20 Hz	`distance_speed_scaler` timer (`_tick_dt = 0.05`).
relay tick	50 Hz	`jtc_servo_relay` interpolation (`rate_hz`).

Note The scaler does not own a topic to the relay — it writes the relay's `live_speed` **parameter** over the `/<relay_node>/set_parameters` service (`relay_node` defaults to `joint_trajectory_controller`, which is the rclpy node name of `jtc_servo_relay`). On stale distance or MANUAL mode it also calls `/<playback_node>/pause` and `/resume` (`std_srvs/Trigger`).

Down-ramping is intentionally **not** rate-limited: a reduction in commanded speed is applied on the next tick.

3.4 Key files

3.4.1 Unity (C#)

`ProximitySpeedController.cs`
Local-only controller. Computes the nearest human-to-`robotBase` distance, applies the EMA filter and asymmetric slew, and exposes `Factor` (0..1). Consumed directly by `PickPlaceLoop`.

`SafetyHud.cs`
Dual-mode. In **ROS mode** it publishes the nearest distance on `/quest/min_distance` (~20 Hz) and subscribes to `/robot/speed_factor` to display what the robot is actually doing. In **local demo** mode it reads distance and factor straight from `ProximitySpeedController`.

`ScalingModeToggle.cs`
AUTO/MANUAL switch. A controller button or a near-touch poke publishes `/quest/scaling_enabled` (Bool), which toggles `distance_speed_scaler.enabled`. When disabled, the manual console owns `live_speed`.

`PickPlaceLoop.cs`
Local demo consumer — multiplies position and rotation motion by `Factor` each frame.

3.4.2 ROS 2 (Python)

`distance_speed_scaler.py`
The authoritative node on the real-robot path. Subscribes `/quest/min_distance`, applies the same filter + slew, publishes `/robot/speed_factor` as a heartbeat, and continuously sets the `jtc_servo_relay.live_speed` parameter. Handles the stale timeout and pauses `/dpp_playback` on stale distance or MANUAL mode.

`jtc_servo_relay.py`
Reads `live_speed` every 50 Hz tick and advances a trajectory-time cursor by `period * live_speed`, then interpolates the keyframes and publishes joint positions to `/servo_node/commands`. Because speed is applied to the **time cursor** rather than to waypoints, it can slow or stop **mid-motion** smoothly.

`dpp_orchestrate.py`
Lab orchestrator that sets MoveIt-level `velocity_scaling` per phase (e.g. normal 0.15, fast 0.50) for dynamic measurement runs — a coarser, waypoint-level scaling distinct from the continuous `live_speed`.

3.5 Where speed is applied

1. **Unity IK-target motion** — `PickPlaceLoop` scales per-frame position/rotation by `Factor`.
2. **Trajectory playback cursor** — `jtc_servo_relay` advances time by `period * live_speed` each tick (continuous, mid-motion).
3. **MoveIt waypoint scaling** — `dpp_playback` `max_velocity_scaling_factor` per waypoint (coarser).
4. **EGM servo loop** — interpolated joint positions are streamed over UDP to the controller.

3.6 Design notes

- The Unity and ROS implementations use an **identical** formula, so the offline demo and the real robot behave the same.

- `/robot/speed_factor` is fed back to the headset HUD so the operator always sees the speed the robot is actually executing, not just the commanded value.
- The MoveSpeed/MaxSpeedDeviation caps in the RAPID source are a **separate, dominant** governance layer — the EGM correction can never exceed the controller-side limits regardless of `live_speed`.

Pinch-and-Move End-Effector Control

4.1 Purpose

The operator controls the robot's end-effector with bare hands. A virtual handle sits on the robot flange; the operator **pinches** it and **drags** it through space. The handle's world pose becomes an IK target, which is solved to joint angles and streamed to the controller. The visible handle stays locked to the real flange, so the operator always grabs "the robot" rather than a free-floating widget.

Two interaction methods exist, both feeding the same world-space IK target:

- **Direct hand-delta grab** (`PhantomGrabRelay`) — the IK target moves by the hand's displacement since the moment of grab.
- **Visual-lock grab** (`IKHandleVisualLock`) — Meta's grab interactable moves a handle, the script copies its pose to the IK target and snaps the visible mesh back to the TCP.

A third, discrete gesture — **spatial pinch tap** — places a target on a real-world surface by raycasting from the palm.

4.2 Data flow

```
Quest hand  --(Meta SDK pinch/grab recogniser)-->  GestureRouter
                                                      |  Pinch -> Teleop
                                                      |  Point -> Jog
                                                      |  Free  -> Command
                                                      v
   +-------------------------------------------------------------+
   | Continuous grab (one of):                                   |
   |   PhantomGrabRelay.LateUpdate                               |
   |     ikTarget.pos = ee_pos_at_grab + (hand - hand_at_grab)*gain|
   |   IKHandleVisualLock.LateUpdate                             |
   |     ikTarget.pos = grabbed_handle.pos ; handle.pos = tcp.pos  |
   +-------------------------------+-----------------------------+
                                   v
   IKTargetPosePublisher.Update  (50 Hz, only while grabbed)
     express target in robotBase frame; convert Unity LH->ROS RH (FLU)
     publish geometry_msgs/PoseStamped -> /metamove/ik_target
                                   |
                                   v ROS-TCP-Connector
   +-------------------------------------------------------------+
   | ROS 2 - one of two relays:                                  |
   |  moveit_ik_relay.py  (position IK):                         |
   |    gate: target fresh? joint_state fresh?                   |
```

```
|    /compute_ik(pose, seed=joint_states)                          |
|    per-joint slew clamp -> /servo_node/commands (Float64MArray)|
|  pose_to_twist_node.py  (servo twist):                           |
|    P-control pos error + quaternion-log rot error                |
|    -> /servo_node/delta_twist_cmds (TwistStamped)                |
+------------------------------+-----------------------------+
                               v
     EGM bridge (Windows) -> UDP -> GoFa controller
```

4.3 Hand tracking & gesture routing

`OVRHandPoseProvider.cs`
: Wraps the Meta hand-tracking rig and exposes palm position, palm normal, per-finger curl, and index-pointing direction in world space.

`GestureRouter.cs`
: Central gesture dispatcher. Meta's shape recognisers raise pinch/point begin/end events here; `UpdateModeFromGesture()` selects the active mode — **pinch -> Teleop**, **point/thumb -> Jog**, **hand free -> Command** — and forwards `OnBegin/OnEnd` to the grab relays and the spatial-pinch controller.

Pinch detection itself is provided by the Meta SDK shape recognition and wired into the router via UnityEvent adapters.

4.4 Continuous teleoperation

4.4.1 Method A — `PhantomGrabRelay.cs` (direct hand-delta)

The visible sphere is a child of `Joint_6` and never moves visually. A separate world-space `ikTarget` transform is driven by the hand:

```
// on grab begin
_ikStartPos  = transform.position;   // current EE position
_handStartPos = anchorPos;           // hand grab-point at grab start

// each frame while grabbed
Vector3 posDelta  = (anchorPos - _handStartPos) * dragGain;
Vector3 targetPos = _ikStartPos + posDelta;
ikTarget.position = targetPos;       // clamped to maxReachM from the EE
```

The hand anchor is taken, in priority order, from the live Oculus selecting point, an inspector override, or the camera as a last resort.

Parameter	Default	Meaning
`dragGain`	1.0	Multiplier on hand displacement (1.0 = 1:1 motion).
`maxReachM`	0.5 m	Clamp on IK-target distance from the current EE.

4.4.2 Method B — `IKHandleVisualLock.cs` (visual lock)

Meta's `HandGrabInteractable` (or `DistanceHandGrabInteractable`) moves a handle while grabbed. Each `LateUpdate` the script copies that pose to the IK target and then snaps the visible handle back to the TCP, so the mesh stays locked to the real flange:

```
if (IsGrabbed) ikTarget.position = transform.position; // hand motion -> IK target
else           ikTarget.position = tcp.position;       // idle: target at EE
transform.position = tcp.position;                      // visible lock to flange
```

4.4.3 Discrete — `SpatialPinchController.cs` (tap-to-place)

A quick pinch-tap (<= `tapMaxDurationSeconds`, default 0.2 s) raycasts from the palm along the palm normal up to a configured length and places a waypoint / IK target at the hit point. Longer pinches fall through to the continuous teleop mode.

4.5 Unity -> ROS publishing

`IKTargetPosePublisher.cs`
: Publishes the IK target at 50 Hz. It expresses the target in the robot base frame, converts Unity's left-handed Y-up coordinates to ROS right-handed Z-up (FLU), and publishes a `geometry_msgs/PoseStamped` on **`/metamove/ik_target`** with `frame_id = base_link`.

Publishing is **gated**: it streams only while the handle is actually grabbed (`onlyWhenGrabbed`), which prevents the robot from chasing a stale target after release.

Field	Value
Topic	`/metamove/ik_target`
Message	`geometry_msgs/PoseStamped`
Frame	`base_link`
Rate	50 Hz

4.6 ROS-side relays

4.6.1 `moveit_ik_relay.py` — position IK

Subscribes `/metamove/ik_target` and `/joint_states`, and calls MoveIt's `/compute_ik` service. The loop runs at 50 Hz with two **fail-safe gates**:

- **Target freshness** — if the last target is older than `target_timeout`, do nothing.
- **Seed freshness** — if `/joint_states` is older than `joint_state_timeout`, refuse to command (no live robot pose -> no motion).

The IK request seeds from the current joint state. The solution is **slew-limited per joint** (`max_joint_speed * dt`) before publishing to `/servo_node/commands`.

Parameter	Default	Meaning
`target_timeout`	0.3 s	Drop stale grab targets.
`joint_state_timeout`	0.5 s	Refuse to command without a fresh seed.
`max_joint_speed`	0.3 rad/s	Per-joint slew clamp (~0.006 rad/tick @ 50 Hz).

4.6.2 `pose_to_twist_node.py` — servo twist (alternative)

Converts an absolute pose target into a Cartesian velocity twist for MoveIt Servo. It reads the current EE pose from TF, applies P-control on the position error and a quaternion-log (axis-angle) term on the orientation error, clamps to maximum linear and angular speed, and publishes `geometry_msgs/TwistStamped` on `/servo_node/delta_twist_cmds`.

Parameter	Default	Meaning
`linear_gain`	1.5 /s	Position P-gain.
`angular_gain`	1.5 /s	Orientation P-gain.
`max_linear`	0.25 m/s	Linear velocity clamp.

`max_angular`	1.0 rad/s	Angular velocity clamp.
`deadband_pos_m`	small	Ignore sub-millimetre jitter.

4.7 Topics & message types

Topic	Message	Producer -> Consumer (rate)
`/metamove/ik_target`	`geometry_msgs /PoseStamped`	Unity publisher -> IK/twist relay (50 Hz)
`/servo_node/commands`	`std_msgs /Float64MultiArray`	`moveit_ik_relay` -> EGM bridge (50 Hz)
`/servo_node /delta_twist_cmds`	`geometry_msgs /TwistStamped`	`pose_to_twist_node` -> MoveIt Servo (100 Hz)
`/joint_states`	`sensor_msgs /JointState`	EGM bridge -> relays (seed)

4.8 Design notes

- The visible handle is always locked to the **real** flange; the operator manipulates a separate invisible IK target. This avoids visual drift between command and reality.
- Both relays enforce **seed freshness** — without a live `/joint_states` they will not command motion, which prevents commanding into an unknown robot state.
- The position-IK relay and the servo-twist node are interchangeable backends; IK gives absolute pose tracking, twist gives smooth velocity control through MoveIt Servo's collision/singularity handling.

Chapter 5

Dashboard and HMI

MetaMove exposes three operator surfaces: a **web dashboard** for monitoring and supervisory control, an **in-headset HMI** for immersive in-world control, and a set of **operator console** scripts for scripted or recovery operations. All three talk to the same ROS 2 graph.

5.1 Web dashboard (MRE2-GOFA_Dashboard)

Note The web dashboard is maintained as a separate component (git submodule `MRE2-GOFA_Dashboard`). It is summarised here because it is part of the operating system of MetaMove.

5.1.1 Technology stack

- **Backend** — FastAPI (Python) joining the ROS 2 graph through `rclpy`. It exposes a REST API plus a WebSocket stream, drives `FollowJointTrajectory` actions and MoveIt Servo twist commands, and persists history to PostgreSQL (or SQLite).
- **Frontend** — HTML/JS/CSS. `index.html` is the monitoring dashboard; `hmi.html` is a tablet-oriented GoFa control surface with a tiled layout. A WebSocket client streams live data.
- **Deployment** — containerised via `deploy/dashboard/dashboard.Dockerfile`; a slim image that joins an existing ROS 2 graph. Served on `:8080` (HTTP) or behind Nginx on `:8443`.

5.1.2 What it shows and controls

Surface	Capability
Live dashboard (`/`)	Joint states, TCP pose, EGM state, 3D digital twin.
GoFa HMI (`/hmi`)	Tiled control: speed, HRC safety, status, maintenance. Axial jog (J1–J6) via hold-to-press; linear/TCP jog via MoveIt Servo; speed gauges and payload; safety zones; maintenance trends, event timeline, toasts; developer view (topic freshness, packet flow, JSON preview).

5.1.3 Connection to the robot

The backend subscribes to `/joint_states`, `/tf`, `/diagnostics`, and `/egm/*`, and sends commands via the `FollowJointTrajectory` action and `/servo_node/delta_twist_cmds`. It auto-discovers ROS topics at runtime and supports both EGM (UDP) and RWS (HTTPS) backends.

Representative API endpoints:

```
GET  /api/snapshot
GET  /api/history/summary?window=1h|24h|7d|30d|90d
GET  /api/history/series
POST /api/ingest
GET  /api/hmi/state
POST /api/hmi/jog/start    * /api/hmi/jog/heartbeat    * /api/hmi/jog/stop
POST /api/hmi/tcp/start    * /api/hmi/tcp/heartbeat
POST /api/hmi/home
```

The jog/TCP endpoints use a **start → heartbeat → stop** pattern: motion continues only while heartbeats arrive, so a dropped connection stops the robot (dead-man behaviour).

5.2 In-headset HMI (Unity Quest 3)

The immersive HMI lives in `unity-quest/Assets/MetaMove/Scripts/UI/`, with `Hud.unity` as the main scene. It is organised in three layers:

1. **L1 — Home**: a hand-anchored radial menu (palm-up gesture) with eight wedges for quick actions (Status, Paths, Position, Safety, Ghost toggle, Envelope toggle, HUD toggle, Robot info).
2. **L2 — Floating panels**: world-positioned panels spawned on demand.
3. **L3 — Physical fixtures**: permanent 3D objects (E-Stop mushroom, pedestal, floor grid).

5.2.1 Core UI scripts

Script	Purpose
`NearTouchButton.cs`	Pinch / finger-near button with dwell-based firing and visual feedback.
`StatusHud.cs`	Curved status HUD (lazy-follow, yaw-only) — battery, link Hz, motors, passthrough.
`MainDashboardPanel.cs`	Hub with tabs (Status / Control / Path / Safety / Motors / Body / Voice / System) and a persistent E-Stop row.
`FlexPendantPanel.cs`	ABB-style teach-pendant emulation: resizable, wrist-attachable, 12 jog buttons (6 axes × 2), E-Stop.
`TelemetryPanel.cs`	Six joint gauges, TCP XYZ / RPY readouts, Hz label.
`ConnectionPanel.cs`	IP / port / mode selection (Real / Virtual / Offline), status LEDs (EGM / RWS / ROS / MoveIt), latency.
`SafetyPanel.cs`	Zone toggles, speed-cap slider, separation distance, ISO mode, E-Stop, stop-reason log.
`PathsPanel.cs`	Waypoint list, path dropdown, add/clear/run, speed slider, loop toggle.
`PrecisePositionPanel.cs`	Six XYZ + RPY sliders with a numpad popup for exact entry.
`PanelManager.cs`	Singleton spawner; registers panels by ID and opens/closes on demand.
`UiModeController.cs`	Switches between Minimal, Control-Center, and FlexPendant UI modes.

In-world overlays (`ShowPose`, `ShowAngles`, `ShowTorque`, `PathPreviewRenderer`, `AxisGizmo`, `DistanceRuler`, …) annotate the robot directly in space.

5.2.2 UI modes

1. **Minimal (default)** — radial menu + on-demand mini-panels + in-world overlays + HUD.
2. **3-panel control center** — three permanent panels around the user (Telemetry, Paths, Safety).

3. **FlexPendant** — a single resizable ABB-style teach pendant (two-hand corner pinch to resize).

All panels use the Meta UI Set components, are grab-translatable (no rotation), and re-orient to the camera on spawn. The curved HUD sits at ~60 cm with status LEDs, TCP pose, link Hz, a mode badge, mini torque bars, and safety status.

5.3 Operator console tools (bridge/egm-bridge)

Command-line tools that connect to `rosbridge` (WebSocket, `:9090`) via `roslibpy`. They are Windows-friendly (use `msvcrt` for key input) and intended for testing, lab runs, and recovery.

Tool	Purpose
`control_console.py`	Unified speed control: toggle **MANUAL** (keys 0–9 / `f` freeze) vs **QUEST** mode (distance-based scaling owns `live_speed`).
`teleop_console.py`	Keyboard teleop for MoveIt Servo: `w/s/a/d/q/e` linear, `i/k/j/l/u/o` angular, `1-9` speed; publishes `/servo_node/delta_twist_cmds`.
`playback_console.py`	Teach-playback control: `p` pause, `r` resume, `1-9/0` speed, space = instant stop; calls `/dpp_playback/pause` and `/resume`.
`relay_speed_console.py`	Direct immediate `live_speed` control (`0`/space freeze, `1-9` %, `f` full) for mid-motion adjustment; also home/pause/resume.
`rescue_jog.py`	Free the robot from servo collision/singularity braking: `--axis 1-6 --deg ±20` publishes ramped joint steps to `/servo_node/commands`, limited to ±20°/invocation.
`activate_servo.py`	Robust MoveIt Servo activation: calls `/servo_node/start_servo`, switches to TWIST command type, monitors `/servo_node/status` (avoids DDS discovery issues).
`inject_command.py`	Publishes a raw `Float64MultiArray` to `/servo_node/commands` for testing.
`dpp_orchestrate_win.py`	Dynamic-measurement orchestration (teach / playback cycles) on Windows.
`egm_bridge_servo.py`	Core EGM/UDP bridge for joint/pose teleoperation.
`rws2_loadmod.py`, `rws2_fix_module.py`	RWS utilities to load / fix RAPID modules.

5.4 Design notes

- The three surfaces are **views onto one ROS graph**, not separate control stacks — the dashboard, the headset HMI, and the consoles all read the same telemetry and write to the same command topics.
- Jog/TCP control uses a **heartbeat / dead-man** pattern: motion only continues while fresh heartbeats arrive, so a lost connection stops the robot.
- `rescue_jog.py` deliberately bypasses servo velocity scaling and is tightly bounded (±20° per call) so it can recover a braked robot without becoming a back door around the safety layer.

Chapter 6

Reference

Consolidated reference for the ROS topics, parameters, and tunables described in the subsystem chapters.

6.1 ROS topics

Topic	Message	Direction	Notes
`/quest/min_distance`	`std_msgs/Float32`	Quest → ROS	Nearest human-to-robot distance (m), ~20 Hz.
`/quest /scaling_enabled`	`std_msgs/Bool`	Quest → ROS	AUTO/MANUAL toggle for the speed scaler.
`/robot/speed_factor`	`std_msgs/Float32`	ROS → Quest	Heartbeat of the applied speed factor (0..1).
`/metamove/ik_target`	`geometry_msgs /PoseStamped`	Quest → ROS	IK target TCP pose in `base_link`, 50 Hz, gated by grab.
`/servo_node /commands`	`std_msgs /Float64MultiArray`	ROS → bridge	Six joint positions (rad).
`/servo_node /delta_twist_cmds`	`geometry_msgs /TwistStamped`	ROS → Servo	Cartesian velocity command.
`/joint_states`	`sensor_msgs /JointState`	bridge → ROS	Live robot state; seed for IK and slew base.
`/diagnostics`, `/tf`, `/egm/*`	—	bridge /MoveIt → dashboard	Telemetry consumed by the dashboard.

6.2 ROS parameters

Node	Parameter	Default	Meaning
`distance_speed_scaler`	`d_near`	0.6 m	Freeze threshold.
`distance_speed_scaler`	`d_far`	2.0 m	Full-speed threshold.
`distance_speed_scaler`	`ema_alpha`	0.3	EMA low-pass strength.
`distance_speed_scaler`	`up_rate`	0.6 /s	Maximum ramp-up rate.
`distance_speed_scaler`	`stale_timeout`	1.5 s	Freeze/pause on stale distance.
`jtc_servo_relay`	`live_speed`	1.0	Continuous speed multiplier (0..1).

moveit_ik_relay	target_timeout	0.3 s	Drop stale grab targets.
moveit_ik_relay	joint_state_timeout	0.5 s	Refuse to command without fresh seed.
moveit_ik_relay	max_joint_speed	0.3 rad/s	Per-joint slew clamp.
pose_to_twist_node	linear_gain	1.5 /s	Position P-gain.
pose_to_twist_node	angular_gain	1.5 /s	Orientation P-gain.
pose_to_twist_node	max_linear	0.25 m/s	Linear velocity clamp.
pose_to_twist_node	max_angular	1.0 rad/s	Angular velocity clamp.

6.3 Network ports

Port	Protocol	Use
6511	UDP	EGM joint path (UDPUC).
6515	UDP	EGM pose path (ROB_1).
9090	WebSocket	rosbridge (operator consoles, dashboard).
10000	TCP	ROS-TCP-Endpoint (Quest client).
443	HTTPS	RWS (controller web services).
8080 / 8443	HTTP/HTTPS	Web dashboard.

6.4 Glossary

EGM
Externally Guided Motion — ABB's low-latency external motion interface over UDP.

RWS
Robot Web Services — the controller's HTTP/REST interface.

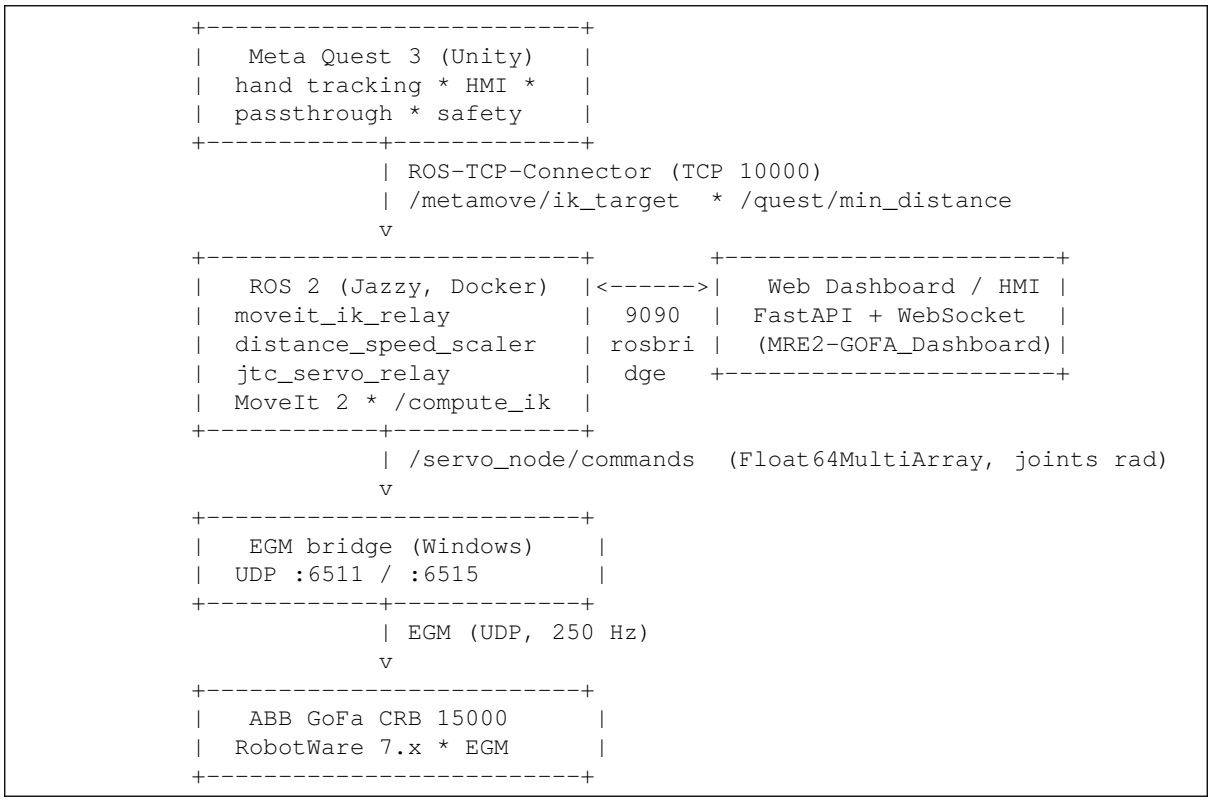
UDPUC
UDP Unicast Communication device on the controller used by EGM; binds to a specific remote address/port.

TCP (robotics)
Tool Center Point — the controlled point at the robot's end-effector (distinct from TCP the network protocol).

FLU
Forward-Left-Up — the ROS right-handed Z-up convention; Unity is left-handed Y-up, so a frame conversion is applied when publishing poses.

Servo
MoveIt Servo — real-time Cartesian/joint jogging with collision and singularity handling.

System overview



The architecture is intentionally **dual-path**:

- **Real robot** — Quest -> ROS 2 -> EGM bridge (Windows) -> GoFa controller over EGM/UDP.
- **Virtual controller / simulation** — the same ROS 2 graph drives a RobotStudio Virtual Controller via RWS where EGM/UDP is not available on the same host.

Repository layout

Path	Contents
`unity-quest/`	Unity project for the Quest 3 client (hand tracking, HMI, safety, ROS publishers).
`ros2/docker/metamove_bridge/`	ROS 2 Python package: IK relay, speed scaler, trajectory relay, playback.
`bridge/egm-bridge/`	Windows EGM/UDP bridge and operator console tools.
`MRE2-GOFA_Dashboard/`	Web dashboard (FastAPI backend + HTML/JS frontend).
`deploy/dashboard/`	Container deployment glue for the dashboard.
`robotstudio/`	RAPID modules and RobotStudio station artifacts.
`docs/`	Architecture notes and this technical documentation.

Chapter 9

How to read this document

Each subsystem chapter follows the same structure: **purpose -> data flow -> key files -> parameters -> notes**. Topic names, message types, and tunable parameters are listed verbatim so they can be cross-referenced against the running system. The Installation & Setup chapter covers hardware/software requirements and bring-up; the Reference chapter consolidates all ROS topics and parameters.

Chapter 10

Team

MetaMove is developed by:

- **Elias Bitsch**
- **Philip Stix**
- **Viktoriia Ovdiienko**